IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Tu primer MLP en Keras

Teoría, práctica y, al final, material de consulta

Sprint 10 · S10_02 · teoría breve (25 min el 17/11), 60 min de práctica en clase y material de consulta

Lo que programaste, Keras lo hace por ti

En S10_01, con NumPy	En Keras
Una capa: `X @ W + b` y `relu`	`Dense(32, activation="relu")`
La red completa: `RedNeuronal`	`keras.Sequential([...])`
`entropia_cruzada`	`loss="binary_crossentropy"`
`paso_gradiente`, con tasa 0,3	`optimizer="adam"`
El bucle de épocas: `entrenar`	`modelo.fit(..., epochs=30)`
`atras`: los gradientes a mano	Automático: TensorFlow los calcula

Keras no cambia la idea: cambia quién escribe el bucle. Lo que no cambia es que tú eliges datos, arquitectura y pérdida.

LOS DATOS · ESCALA Y CATEGORÍAS

Las redes son sensibles a la escala

Si una columna va de 0 a 400 y otra de 1 a 4, la primera «grita» más que la segunda.

Qué	Ejemplo	Transformación
Numérica	antelación de 150 días; media 100 y desviación 50	**StandardScaler**: (150 – 100) / 50 = **+1,0**
Categórica	canal = agencia, entre web, agencia y teléfono	**OneHotEncoder**: web 0 · agencia 1 · teléfono 0
Fuera	el identificador, las fechas en bruto y el objetivo	No entran en la red

Por qué estandarizar

Con media 0 y desviación 1, **todas las columnas pesan parecido** y el entrenamiento es más estable.

Por qué one-hot

Una categoría no es un número: se reparte en **una columna 0/1 por valor**, sin imponer un orden.

LOS DATOS · SEPARARLOS

Entrenamiento, validación y prueba

60 %
Entrenamiento
600 de cada 1 000: la red ajusta sus pesos

20 %
Validación
200: decidir

20 %
Prueba
200: una sola vez

Fuga de datos

Si el escalador viera la validación o la prueba, «espiaríamos» datos que la red no debe conocer.

Cómo se evita

El escalador se **ajusta solo con el entrenamiento** y después se aplica a validación y prueba.

La validación sirve para decidir (épocas, arquitectura); la prueba se toca una sola vez, al final.

EL MODELO · UN MLP

Dos capas ocultas y una salida

Capa	forma	parámetros
Oculta 1 (ReLU)	10 → 8	10 · 8 + 8 = 88
Oculta 2 (ReLU)	8 → 4	8 · 4 + 4 = 36
Salida (sigmoide)	4 → 1	4 + 1 = 5
Total		**129**

Parámetros de una capa

entradas · neuronas **+ un sesgo por neurona**.

Salida sigmoide + entropía cruzada: la pareja que programaste a mano.

Se construye con **Sequential** y capas **Dense**, y **compile** elige **optimizador, pérdida y métricas**.

EL MODELO · ENTRENAR CON FIT

Época, lote y validación

Época

Una pasada completa por las reservas de entrenamiento. Ejemplo: **1 000** filas.

Lote (batch)

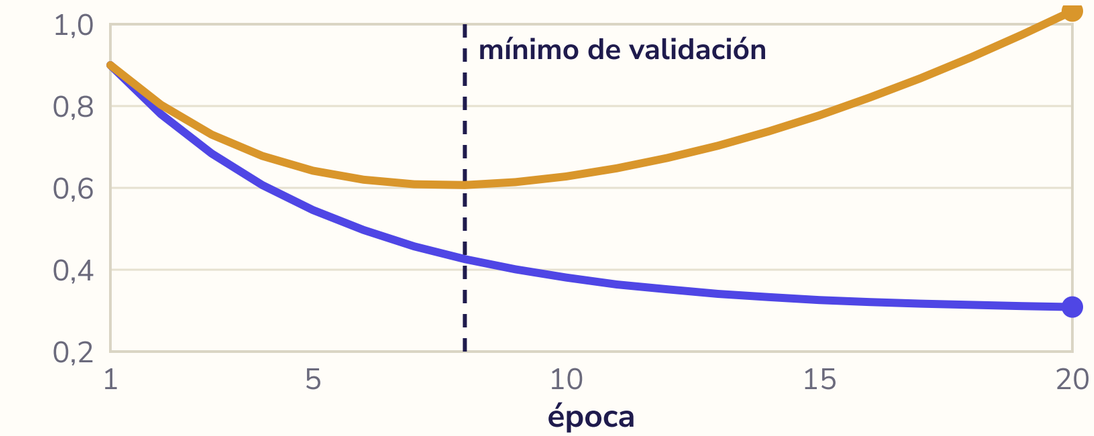
Filas antes de actualizar los pesos. Con lote de **50**: **20** actualizaciones por época.

Validación

En cada época se mide también con datos de validación, **sin entrenar** con ellos.

10 épocas × 20 lotes = 200 actualizaciones. Keras guarda la pérdida y las métricas de cada época para dibujar las curvas.

Lee las curvas: ¿cuándo empieza a memorizar?



Entrenamiento

Sigue **bajando siempre**: la red ajusta cada vez mejor lo que ya ha visto.

Validación

Baja, toca fondo y **vuelve a subir**: ahí empieza a memorizar.

Curvas que se separan = sobreajuste. Se puede parar en el mínimo de validación.

EVALUAR · UNA REFERENCIA SENCILLA

¿Gana la red a un modelo sencillo?

Modelo (ejemplo inventado)	AUC	Exactitud
Siempre «no cancela»	0,500	0,60
Regresión logística	**0,780**	0,71
Red (MLP)	0,795	0,72

Una cifra sola no dice nada

Acertar un 60 % es lo que consigue quien no sabe nada: la red tiene que superar a un modelo **sencillo**.

Ganar por poco es ruido

Con una ventaja de 0,015 de AUC no se justifica un modelo más complejo: se exige un **margen** (por ejemplo 0,02).

El AUC se calcula con las **probabilidades**, no con las clases: mide cómo ordena el modelo por riesgo.

OTRA TAREA · REGRESIÓN

Precio por noche: cambia solo la cabeza

	Clasificación	Regresión
Salida	`Dense(1, "sigmoid")`	`Dense(1)`, sin activación
Pérdida	`binary_crossentropy`	`mse` (métrica legible: `mae`)
Objetivo	0 o 1	precio estandarizado; se deshace para dar euros
Entradas	todas, menos el objetivo	todas, menos el precio (es el objetivo)

MAE

Error medio absoluto: con un MAE de 10 €, el precio previsto se equivoca, **de media, 10 euros por noche**.

Deshacer la escala

Se entrena con el precio estandarizado; para dar euros se vuelve atrás: **v · desviación + media**.

La referencia de «no saber nada» es predecir siempre el precio medio.

CAPACIDAD · MÁS ANCHA, ¿MEJOR?

Más parámetros: más capacidad y más memoria

Red (10 entradas)	parámetros	Qué puede hacer
Pequeña (8, 4)	129	Dibuja fronteras sencillas
Ancha (64, 32)	2 817	Dibuja fronteras muy complejas… y puede **memorizar**

21,8 veces más parámetros

La red ancha tiene más capacidad, pero **memoriza antes**: el entrenamiento mejora y la validación puede empeorar.

Cómo decidir

Se mira la curva de **validación**, no la de entrenamiento, y se compara con una red pequeña.

Más parámetros no son un mejor modelo: solo ayudan si hay datos suficientes.

PRÁCTICA · CUADERNO S10_02

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación	Cargar las herramientas, las reservas y crear **mes_llegada**	Clase
Ejercicio 1	Columnas numéricas y categóricas; separar y preparar los datos	Clase
Ejercicio 2	Construir el MLP de clasificación; entrenar y leer las curvas	Clase
Ejercicio 3	¿Gana el MLP a la regresión logística?	Clase
Ejercicio 4	Construir el MLP de regresión del precio	Casa
Ejercicio 5	El error en euros y la mejora sobre la media	Casa
Mini-reto	Una red más ancha (128-64)	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el ✔**. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

Cómo está ordenado

En el **mismo orden que el cuaderno**: ejercicios 1 a 5 y mini-reto, con su código y sus aclaraciones.

Ejercicio	Tema	Diapositivas
1	Columnas y separar los datos	13, 14 y 15
2	MLP de clasificación y curvas	16, 17 y 18
3	Evaluar y comparar con la logística	19, 20 y 21
4	MLP de regresión	22 y 23
5	Error en euros y mejora	24, 25 y 26
Mini-reto	Una red más ancha	27 y 28

CONSULTA · EJERCICIO 1

Columnas numéricas y categóricas (ejercicio 1)

```
cols_num = ["antelacion_dias", "noches", "adultos", "ninos", "peticiones_especiales",
            "cliente_repetidor", "cancelaciones_previas", "mes_llegada"]
cols_cat = ["tipo_habitacion", "regimen", "canal", "pais_origen", "tarifa", "deposito"]
```

8

columnas **numéricas**

6

columnas **categóricas**

Fuera de la red: id_reserva, las fechas en bruto y cancelada (el objetivo).

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer la lista de columnas (ejercicio 1)

cols_num y cols_cat

8 numéricas (con **mes_llegada**) y 6 categóricas. Un nombre mal escrito (**noche** en vez de **noches**) da error.

Fuera: cancelada e id_reserva

cancelada es **lo que queremos predecir**: como entrada, la red haría trampa. **id_reserva** es solo un número: no explica nada y permitiría memorizar.

Numéricas

Se **estandarizan** (media 0, desviación 1) para que ninguna «grite» más por su escala.

Categóricas

Se pasan a columnas 0/1 (**one-hot**): una columna por categoría.

CONSULTA · EJEMPLO · SEPARAR LOS DATOS

Separar los datos: 3 840, 960 y 1 200

64 %
Entrenamiento
3 840 reservas: la red ajusta sus pesos

16 %
Validación
960: decidir

20 %
Prueba
1 200: una sola vez

```
A_ent = pre.fit_transform(X_ent)    # solo con entrenamiento
A_val = pre.transform(X_val)
A_pru = pre.transform(X_pru)
```

Fuga de datos

Si el escalador viera la validación o la prueba, «espiaríamos» datos que la red no debe conocer.

La validación sirve para decidir (épocas, arquitectura); la prueba se toca una sola vez, al final.

CONSULTA · EJERCICIO 2

Construir el MLP de clasificación (ejercicio 2)

Capa	forma	parámetros
Dense(32, relu)	35 → 32	35 · 32 + 32 = 1 152
Dense(16, relu)	32 → 16	32 · 16 + 16 = 528
Dense(1, sigmoid)	16 → 1	16 + 1 = 17
Total		**1 697**

Parámetros de una capa

entradas · neuronas **+ un sesgo por neurona**. Si el total no cuadra, la forma no es la pedida.

Salida sigmoide + entropía cruzada: la pareja que programaste a mano.

```
modelo = keras.Sequential([keras.layers.Input(shape=(35,))])
modelo.add(keras.layers.Dense(32, activation="relu"))
modelo.add(keras.layers.Dense(16, activation="relu"))
modelo.add(keras.layers.Dense(1, activation="sigmoid"))
modelo.compile(optimizer="adam", loss="binary_crossentropy",
               metrics=[keras.metrics.AUC(name="auc"), "accuracy"])
```

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer el MLP de clasificación (ejercicio 2)

1 697 parámetros

Cada capa: entradas · neuronas **+ un sesgo por neurona**.
Primera: 35 · 32 + 32 = 1 152; luego 528 y 17.

Salida **Dense(1, sigmoid)**

Clasificación binaria: **una** neurona cuya salida (0 a 1) es la probabilidad de cancelar.

compile

Prepara el entrenamiento: optimizador **adam**, pérdida **binary_crossentropy** y métricas **AUC** y **accuracy**.

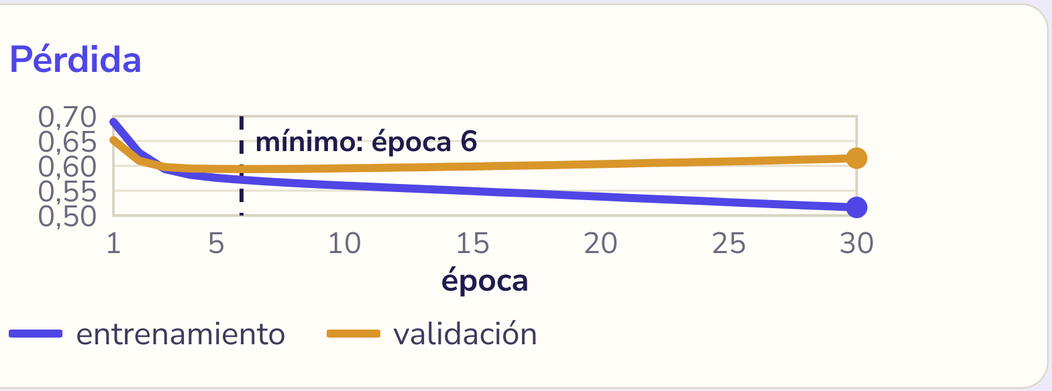
for unidades in capas

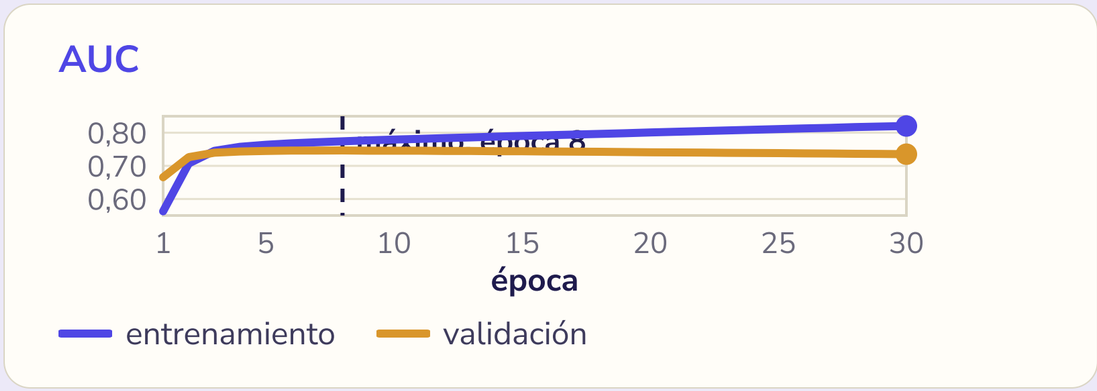
La función recibe las capas como lista: con **capas=(32, 16)** salen 2 ocultas + 1 de salida = **3 capas Dense**.

CONSULTA · EJEMPLO · ENTRENAR CON FIT Y LEER LAS CURVAS

Entrenar 30 épocas y leer las curvas

```
hist_c = mlp_c.fit(Xc_ent, yc_ent,
                   validation_data=(Xc_val, yc_val),
                   epochs=30, batch_size=64, verbose=0)
```





La pérdida de entrenamiento sigue bajando (0,689 → 0,516), pero la de validación toca fondo en la época 6 (0,593) y sube hasta 0,615: sobreajuste.

CONSULTA · EJEMPLO · EVALUAR EN PRUEBA

Evaluar en prueba y comparar

Modelo (1 200 reservas de prueba)	AUC	Exactitud
Siempre «no cancela»	0,500	0,541
Regresión logística	**0,746**	0,678
MLP (32-16)	0,727	0,661

Una cifra sola no dice nada

Acertar un 54 % es lo que consigue quien no sabe nada: la red tiene que superar a un modelo **sencillo**.

Con estos datos `mlp_gana` es `False`: la red queda unas 2 centésimas de AUC por debajo de la logística.

CONSULTA · EJERCICIO 3

¿Gana el MLP a la regresión logística? (ejercicio 3)

```
auc_mlp = roc_auc_score(yc_pru, prob_mlp)    # 0.727
auc_log = roc_auc_score(yc_pru, prob_log)    # 0.746
mlp_gana = bool(auc_mlp - auc_log > 0.02)    # False
```

0,727
AUC del MLP

0,746
AUC logística

False
mlp_gana

La red queda unas 2 centésimas por debajo de la logística: no justifica su complejidad.

CONSULTA · ACLARACIONES · EJERCICIO 3

Leer la comparación (ejercicio 3)

prob_mlp, no clases

El AUC mide cómo **ordena** el modelo las reservas por riesgo; con solo 0/1 se pierde esa información.

bool(... > 0.02)

mlp_gana debe ser **True** o **False**, no la diferencia: se guarda un booleano.

0,727 y 0,746

auc_mlp queda **0,02 por debajo** de **auc_log**: **mlp_gana** es **False**.

Margen de 0,02

Diferencias muy pequeñas son **ruido**: no basta ganar por una milésima para justificar un modelo más complejo.

CONSULTA · EJERCICIO 4

Construir el MLP de regresión (ejercicio 4)

```
def construir_mlp_regresion(n_entradas, capas=(32, 16)):
    modelo = keras.Sequential([keras.layers.Input(shape=(n_entradas,))])
    for unidades in capas:
        modelo.add(keras.layers.Dense(unidades, activation="relu"))
    modelo.add(keras.layers.Dense(1))    # sin activación
    modelo.compile(optimizer="adam", loss="mse", metrics=["mae"])
```

≈ 0,29

MAE de entrenamiento, de 0,68 a

≈ 0,33

MAE de validación, de 0,59 a

Cambian solo la salida, la pérdida y la métrica.

CONSULTA · ACLARACIONES · EJERCICIO 4

Leer el MLP de regresión (ejercicio 4)

Dense(1) sin activación

El precio puede valer **cualquier número**; una sigmoide lo limitaría a 0–1.

loss="mse" y **metrics=["mae"]**

Cambian la **pérdida** y la **métrica**; el resto de la función es igual que en clasificación.

MAE de 0,29 y 0,33

En escala **estandarizada**: la validación queda algo por encima del entrenamiento, un **pequeño sobreajuste**.

esc y **desesc**

precio_noche es el objetivo, así que **no entra** como variable. **desesc** vuelve a euros: **v · desv_p + media_p**.

CONSULTA · EJEMPLO · EVALUAR LA REGRESIÓN

Error medio en euros frente a las referencias

Error medio absoluto en prueba	MAE (€)
Siempre el precio medio	25,52
Regresión lineal (Ridge)	11,87
MLP (32-16)	**11,81**

54 %

menos error que predecir siempre el precio medio (**mejora_vs_media** = 0,537).

Ridge y el MLP empatan

Por 0,06 € de diferencia: la red **no aporta nada** frente a una regresión lineal.

CONSULTA · EJERCICIO 5

El error en euros y la mejora (ejercicio 5)

```
mae_mlp_euros = mean_absolute_error(yr_pru, pred_mlp)                    # 11.81
mae_base = mean_absolute_error(yr_pru, np.full(len(yr_pru), yr_ent.mean()))    # 25.52
mejora_vs_media = 1 - mae_mlp_euros / mae_base                           # 0.537
```

11,81 €

mae_mlp_euros

0,537

mejora_vs_media

El error baja un **54 %** respecto a predecir siempre la media.

CONSULTA · ACLARACIONES · EJERCICIO 5

Leer el error en euros (ejercicio 5)

mae_base

Error de predecir **siempre el precio medio de entrenamiento**: 25,52 €. Es la referencia de «no saber nada».

mean_absolute_error(yr_pru, pred_mlp)

Se compara con **yr_pru en euros**, no con el precio estandarizado: sale 11,81 €.

1 - mae_mlp_euros / mae_base

El cociente es la fracción del error que **queda**; uno menos esa fracción es la que se **elimina**: 0,537, un 54 %.

¿Demuestra que la red hace falta?

No: Ridge consigue casi lo mismo (11,87 €). La mejora es sobre la media, no sobre un modelo lineal.

CONSULTA · MINI-RETO

¿Es mejor la red grande? (mini-reto)

Tras 15 épocas	parámetros	AUC entrenamiento	AUC validación	pérdida validación
(32, 16)	1 697	0,791	**0,744**	**0,599**
(128, 64)	12 929	0,849	0,724	0,637

```
mlp_ancho = construir_mlp_clasificacion(n, capas=(128, 64))
hist_ancho = mlp_ancho.fit(..., epochs=15, batch_size=64)
auc_ancho = hist_ancho.history["val_auc"][-1]   # 0.724
```

7,6 veces más parámetros

Más capacidad: **memoriza antes**. El AUC de entrenamiento sube y el de validación baja.

Más parámetros no son un mejor modelo: la red grande no mejora la validación.

CONSULTA · ACLARACIONES · MINI-RETO

Leer la red ancha (mini-reto, casa)

capas=(128, 64)

12 929 parámetros frente a 1 697: **7,6 veces más**. Sin ese cambio, el **assert** de parámetros falla.

epochs=15

El **assert** exige 15 épocas: la comparación con la red pequeña es a igualdad de entrenamiento.

0,724

La red ancha **no mejora**: su AUC de validación llegó a 0,751 en la época 2 y luego bajó, mientras el de entrenamiento subía hasta 0,85.

Capacidad

Más parámetros = más capacidad, pero también **más facilidad para memorizar**. Solo ayuda con datos suficientes y sobreajuste controlado.